

**SISTEM PENDUKUNG KEPUTUSAN (DSS) PENENTUAN JUARA
LOMBA BERBASIS WEIGHTED DIRECTED GRAPH
MENGUNAKAN ALGORITMA DIJKSTRA**

Dosen Pengampu : I Made Gede Sri Artha, S.T.



Disusun Oleh :

- | | | |
|----|---------------------------|------------------|
| 1. | Kadek Satya Giri Sardhula | NIM : 2501010088 |
| 2. | Malvaraina Nursalim | NIM : 2501010084 |
| 3. | Pande Putu Vidya Reswara | NIM : 2501010064 |

**PROGRAM STUDI INFORMATIKA
INSTITUT BISNIS DAN TEKNOLOGI INDONESIA**

2026

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Proses evaluasi dan pengambilan keputusan untuk menentukan pemenang dalam sebuah perlombaan sering kali dihadapkan pada tantangan objektivitas. Penyelenggara atau juri dituntut untuk menilai setiap peserta secara adil berdasarkan multi-kriteria yang memiliki bobot tingkat kepentingan berbeda, seperti kemampuan penyampaian materi, ide inovasi, hingga estetika penampilan panggung. Jika penentuan juara hanya mengandalkan tabulasi spreadsheet konvensional atau perhitungan rata-rata linear manual, risiko terjadinya kesalahan manusia (*human error*) dalam memasukkan data serta bias penilaian subjektif akan menjadi sangat besar. Cara-cara lama ini juga dinilai kurang transparan karena tidak mampu memetakan klasifikasi tingkatan performa peserta secara mendalam kepada pihak luar atau peserta itu sendiri.

Untuk mengatasi permasalahan tersebut, dibutuhkan sebuah Sistem Pendukung Keputusan (*Decision Support System* atau DSS) yang mampu memodelkan proses penilaian secara lebih transparan, dinamis, dan terstruktur. Dalam bidang ilmu komputer, salah satu struktur data non-linear yang sangat andal untuk merepresentasikan hubungan atau alur relasional berantai dari suatu masalah nyata adalah Graph (Graf). Dalam konteks perlombaan, tahapan penilaian dapat dimodelkan sebagai sebuah jaringan relasional yang terdiri dari pos-pos kriteria penilaian. Alur evaluasi setiap peserta dianalogikan sebagai sebuah perjalanan melintasi pos-pos tersebut, dimulai dari pintu masuk (*Start*) hingga gerbang keputusan (*Finish*).

Pada project akhir ini, pemodelan keputusan dikembangkan menggunakan pendekatan Multi-Layer Directed Graph Berorientasi Cabang (Multi-Path Layered Graph). Di setiap pos kriteria, sistem menyediakan cabang tingkatan kualifikasi performa kompetensi, yaitu *Excel (Excellent)*, *Good*, dan *Avg (Average)*. Nilai asli pencapaian peserta akan dikonversi menjadi sebuah beban biaya penalti (*cost penalty*). Melalui penerapan algoritma pencarian rute terpendek Dijkstra yang dikombinasikan dengan pembatasan jalur ekstrem, sistem dipaksa melakukan analisis untuk mencari rute lintasan dengan total biaya paling minimum (*shortest path*). Peserta yang berhasil menembus jalur kompetensi tertinggi dengan total biaya hambatan paling kecil secara otomatis akan direkomendasikan sebagai Juara 1, sehingga melahirkan keputusan yang sangat objektif dan akurat berbasis data relasional.

1.2 Rumusan Masalah

Berdasarkan latar belakang masalah yang telah dipaparkan, maka rumusan masalah dalam project akhir ini adalah sebagai berikut:

1. Bagaimana cara mentransformasikan nilai kompetensi multi-kriteria peserta lomba ke dalam bentuk arsitektur data *Multi-Path Layered Graph*?

2. Bagaimana memformulasikan nilai asli peserta menjadi bobot beban penalti (*cost penalty*) dan memblokir jalur yang tidak valid menggunakan nilai bobot ekstrem?
3. Bagaimana mengimplementasikan Algoritma Dijkstra dalam melakukan pelacakan lintasan minimum (*shortest path analysis*) untuk menghasilkan keputusan urutan ranking juara lomba yang valid?

1.3 Tujuan

Adapun tujuan yang ingin dicapai melalui pengerjaan project akhir ini adalah:

1. Mendesain model graf sekuensial bercabang yang merepresentasikan lapisan tingkatan kompetensi (*Excel, Good, Avg*) dari kriteria penilaian lomba secara sistematis.
2. Menerapkan Algoritma Dijkstra berbantuan struktur data *Min-Priority Queue* (heapq) pada bahasa Python untuk memproses optimasi pencarian rute juara secara otomatis.
3. Membangun aplikasi DSS interaktif satu halaman menggunakan framework Streamlit yang menyediakan dwi-metode input data (Ketik Manual dan Upload CSV) serta visualisasi diagram horizontal yang informatif.

1.4 Manfaat

Project akhir ini diharapkan dapat memberikan manfaat yang luas, antara lain:

1. Bagi Pengguna (Juri Lomba): Mempermudah juri dalam memproses data nilai peserta dalam jumlah banyak secara instan, adil, transparan, serta adaptif karena persentase bobot kriteria dapat diubah kapan saja secara *real-time*.
2. Bagi Peserta Lomba: Memberikan keterbukaan informasi evaluasi melalui visualisasi alur lintasan yang berhasil dilewati, sekaligus mendapatkan umpan balik analitik pintar (*AI Smart Feedback*) mengenai kelemahan terbesar yang harus diperbaiki.
3. Bagi Mahasiswa: Berfungsi sebagai sarana implementasi praktis untuk menguji pemahaman teori abstrak materi struktur data graf dan kompleksitas algoritma *shortest path* ke dalam program aplikasi solusi dunia nyata.

BAB 2 DASAR TEORI

2.1 Struktur Data Graph

Graph (Graf) adalah struktur data non-linear yang digunakan untuk merepresentasikan koleksi objek-objek yang memiliki hubungan relasional spesifik satu sama lain. Secara matematis, sebuah graf didefinisikan sebagai pasangan set $G = (V, E)$, di mana V mewakili kumpulan *Vertices* (Node/Titik status) dan E mewakili kumpulan *Edges* (Sisi/Garis penghubung antar-node).

Di dalam implementasi sistem ini, struktur data graf dirancang menggunakan dua karakteristik utama:

- **Directed Graph (Graf Berarah)**

Yaitu jenis graf yang setiap sisinya memiliki arah orientasi penunjuk yang jelas. Sifat berarah ini sangat penting untuk menjamin alur evaluasi keputusan bergerak maju secara sekuensial selangkah demi selangkah dari kriteria awal hingga kriteria akhir, serta mencegah adanya perulangan alur yang tidak logis (*looping back*).

- **Weighted Graph (Graf Berbobot)**

Yaitu jenis graf yang setiap sisinya dilekatkan suatu nilai numerik tertentu (*weight*). Dalam project ini, nilai bobot pada sisi melambangkan biaya penalti (*cost penalty*). Jika performa nilai peserta buruk, maka biaya penalti sisinya akan membesar, yang berarti jarak lintasan grafnya menjadi semakin jauh untuk dilewati.

2.2 Decision Support System (DSS)

Decision Support System (DSS) atau Sistem Pendukung Keputusan adalah sebuah sistem berbasis komputer interaktif yang membantu pengambil keputusan (manusia) memanfaatkan data dan model matematika untuk menyelesaikan masalah yang bersifat semi-terstruktur atau tidak terstruktur. DSS mengintegrasikan kemampuan komputasi komputer yang cepat dengan pertimbangan subyektif manusia (seperti penentuan persentase bobot oleh juri) untuk menghasilkan alternatif rekomendasi keputusan terbaik. Karakteristik utama DSS yang baik adalah fleksibilitas antarmuka, transparansi proses kalkulasi model, serta kecepatan dalam memproses perubahan data (*on-the-fly computation*).

2.3 Algoritma Graph yang Digunakan (Algoritma Dijkstra)

Algoritma utama yang digunakan sebagai mesin pengambil keputusan di dalam aplikasi DSS ini adalah Algoritma Dijkstra. Algoritma Dijkstra termasuk ke dalam kategori algoritma *greedy* yang digunakan untuk memecahkan masalah lintasan terpendek dari satu titik sumber menuju titik-titik lainnya (*single-source shortest path problem*) pada graf berarah berbobot dengan syarat nilai bobot sisi tidak boleh negatif (**weight** ≥ 0).

Mekanisme kerja umum algoritma ini didasarkan pada strategi pelacakan sebagai berikut:

1. Menentukan titik awal perjalanan sebagai node aktif dan mengatur nilai jarak sementara menuju dirinya sendiri bernilai 0, sedangkan jarak ke semua node lainnya diatur bernilai tak terhingga (∞).
2. Memeriksa semua node tetangga yang terhubung langsung dengan node aktif, lalu melakukan proses pembaruan jarak kumulatif (*Relaxation*). Jika jarak baru yang dihitung lebih kecil dari jarak sementara yang tercatat, maka sistem akan memperbarui nilai jarak tersebut dan mencatat node aktif sebagai jalur leluhurnya (*parent node*).
3. Memilih node yang belum dikunjungi dengan nilai jarak kumulatif terkecil untuk dijadikan sebagai node aktif berikutnya.
4. Mengulangi proses tersebut hingga node tujuan berhasil dicapai dan ditandai sebagai selesai.

Untuk mengoptimalkan efisiensi pencarian node dengan jarak terkecil pada setiap iterasi, kode program memanfaatkan struktur data Min-Priority Queue melalui pustaka bawaan Python `heapq` (Min-Heap). Struktur data priority queue ini menjamin operasi pengambilan nilai minimum (*extract-min*) dapat berjalan dengan kecepatan komputasi yang sangat cepat, sehingga proses analisis keputusan pada aplikasi DSS dapat dilakukan secara instan.

BAB 3

ANALISIS DAN PERANCANGAN

3.1 Analisis Masalah

Sistem Pendukung Keputusan (DSS) ini dirancang untuk menyelesaikan tantangan objektivitas juri dalam mengevaluasi peserta lomba berdasarkan kriteria majemuk dinamis, yaitu Public Speaking, Kreativitas, dan Penampilan. Pada metode konvensional, akumulasi nilai dilakukan menggunakan penjumlahan linear atau rata-rata biasa. Kendala utama dari pemodelan matematika tersebut adalah kurangnya transparansi mengenai klasifikasi tingkatan kompetensi peserta di setiap pos kriteria.

Untuk menyelesaikan masalah ini, sistem mentransformasikan penilaian ke dalam bentuk pencarian lintasan optimum menggunakan Algoritma Dijkstra. Kendala logis yang harus dipecahkan adalah fakta bahwa algoritma Shortest Path (seperti Dijkstra) bekerja mencari bobot terkecil (minimum cost), sementara penilaian lomba mencari nilai terbesar. Masalah ini diselesaikan secara komprehensif melalui dua tahapan analisis logika fungsional:

1. Klasifikasi Kategori Kompetensi (Node Layering)

Nilai mentah peserta (skala 0–100) dikelompokkan secara otomatis ke dalam tiga tingkat performa (cabang kompetensi):

- **Excel (Excellent):** Untuk nilai ≥ 85
- **Good:** Untuk nilai $70 \leq \text{nilai} < 85$
- **Avg (Average):** Untuk nilai < 70

2. Formulasi Biaya Penalti (Cost Penalty Function)

Untuk mencerminkan performa asli peserta ke dalam bobot graf (edge weight), nilai dikonversi menjadi penalti jarak. Jika nilai peserta tinggi di kelasnya, bobot hambatan perjalanannya akan kecil. Rumus matematis yang dirancang adalah:

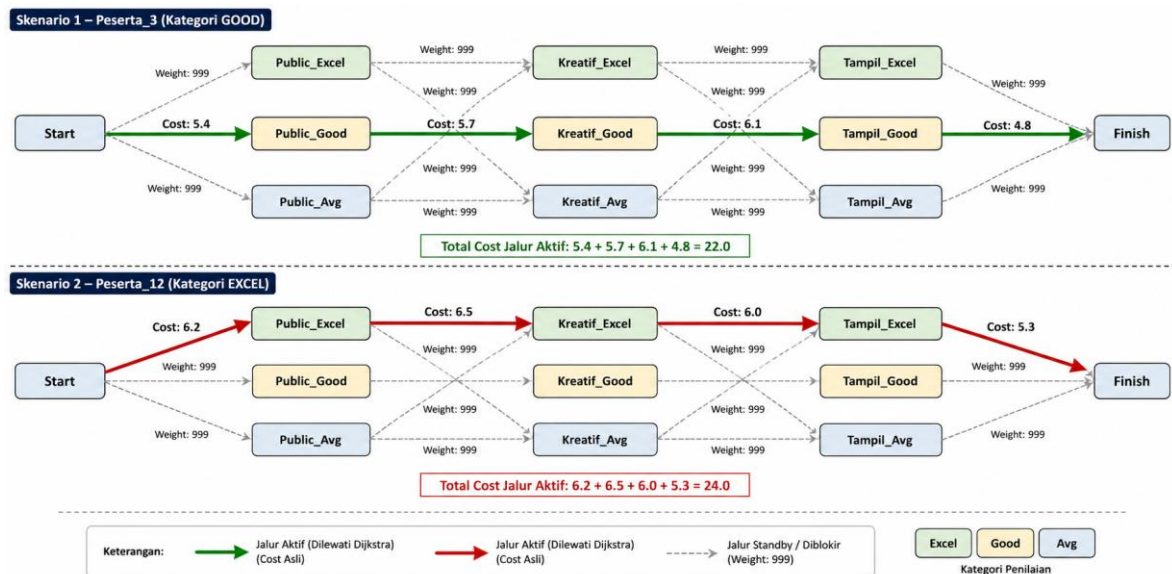
$$\text{Bobot Sisi (Cost)} = (100 - \text{Nilai Peserta}) \times \left(\frac{\text{Bobot Kriteria}}{100} \right)$$

Sebaliknya, jika peserta tidak memenuhi kualifikasi kelas kriteria tertentu (misalnya, seorang peserta masuk kategori Good namun sistem memeriksa jalur Excel), sistem akan memberikan **nilai penalti ekstrem (banned path cost) sebesar 999.0**. Bobot ekstrem ini berfungsi memblokir gerbang jalur tersebut sehingga mesin Dijkstra dipaksa berbelok hanya melewati cabang kompetensi yang sesuai dengan kemampuan riil peserta.

3.2 Desain Graph

Model graf yang dirancang dalam sistem ini tidak lagi berupa rantai lurus linear tunggal, melainkan beralih menggunakan struktur **Multi-Layer Directed Graph**

Berorientasi Cabang (Multi-Path Layered Graph). Model ini memetakan seluruh kemungkinan kombinasi keputusan yang dapat dilalui oleh semua peserta di dalam satu cetak biru arsitektur graf.



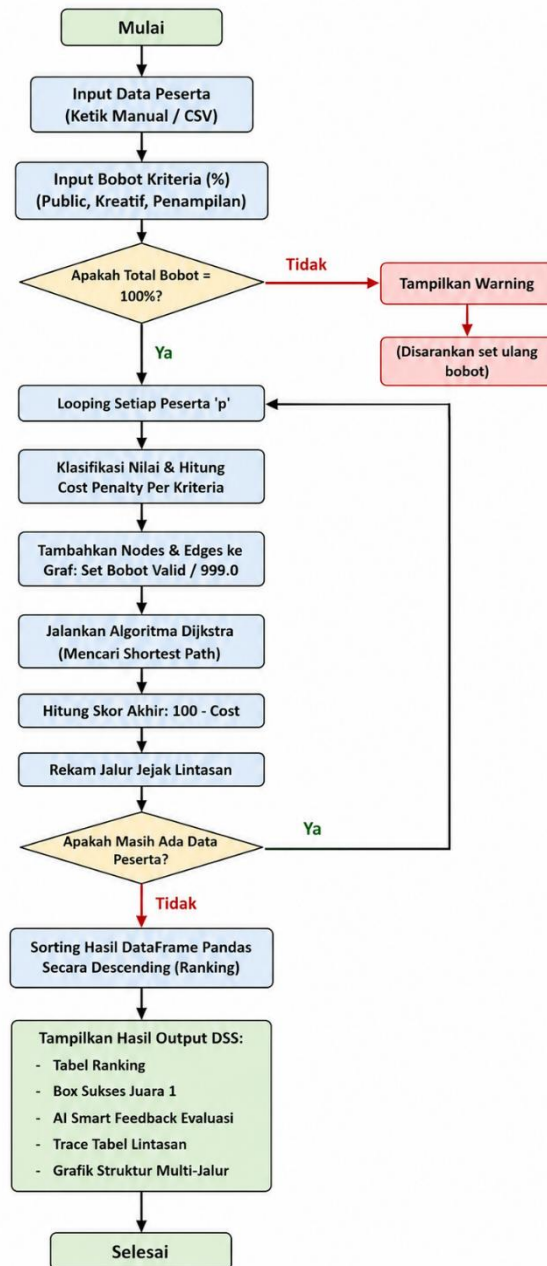
Struktur arsitektur graf ini terdiri dari 5 lapisan utama (*layers*):

1. **Layer Sumber:** Diwakili oleh satu node awal tunggal, yaitu Start.
2. **Layer Kriteria 1 (Public Speaking):** Terbagi menjadi 3 node cabang (Public_Excel, Public_Good, Public_Avg).
3. **Layer Kriteria 2 (Kreativitas):** Terbagi menjadi 3 node cabang (Kreatif_Excel, Kreatif_Good, Kreatif_Avg).
4. **Layer Kriteria 3 (Penampilan):** Terbagi menjadi 3 node cabang (Tampil_Excel, Tampil_Good, Tampil_Avg).
5. **Layer Target:** Diwakili oleh satu node akhir tujuan, yaitu Finish.

Setiap node di satu layer terhubung sepenuhnya menuju seluruh node di layer berikutnya (*Fully Connected Layer Transitions*). Jalur riil yang diambil oleh peserta akan menyala secara visual berdasarkan hasil kalkulasi *Shortest Path* Dijkstra, memisahkan rute efisien (warna hijau/tebal) dengan rute yang tidak valid (warna abu-abu/tipis).

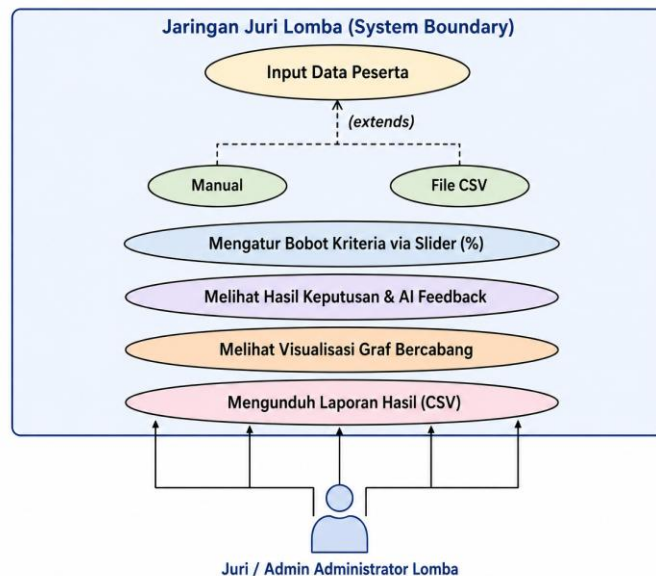
3.3 Flowchart

Berikut adalah diagram alur (Flowchart) yang menggambarkan logika eksekusi sistem dari pemrosesan data mentah hingga penentuan rekomendasi keputusan akhir:



3.4 Use Case Diagram

Use Case Diagram ini menggambarkan interaksi fungsionalitas antara pengguna (user) dengan fungsionalitas di dalam aplikasi DSS. Aktor utama dalam sistem ini adalah **Juri / Administrator Lomba**.



3.5 Struktur Node dan Edge

Untuk menjamin keakuratan proses pencatatan lintasan terpendek, arsitektur data graf dalam sistem dideklarasikan menggunakan susunan pengelompokan komponen *Vertices* (Node) dan *Edges* (Sisi) yang kaku dan terstruktur.

A. Komponen Node (Vertices)

Sistem memiliki 11 Node Utama yang bertindak sebagai pos-pos persinggahan evaluasi keputusan:

1. Start: Node gerbang awal perjalanan evaluasi seluruh peserta.
2. Public_Excel, Public_Good, Public_Avg: Node cabang layer kriteria *Public Speaking*.
3. Kreatif_Excel, Kreatif_Good, Kreatif_Avg: Node cabang layer kriteria *Kreativitas*.
4. Tampil_Excel, Tampil_Good, Tampil_Avg: Node cabang layer kriteria *Penampilan*.
5. Finish: Node target terminal akhir untuk mencatat total biaya perjalanan.

B. Komponen Sisi (Edges) dan Logika Pembobotan

Hubungan antar-node di dalam sistem diatur menggunakan aturan pembobotan bersyarat (*conditional weight assignment*) untuk membatasi pergerakan algoritma Dijkstra agar sesuai dengan profil riil peserta:

Node Asal (Source)	Node Tujuan (Target)	Aturan Validasi Nilai & Penetapan Bobot (Edge Weight)
Start	Public_Excel	Jika nilai Public $\geq 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Start	Public_Good	Jika nilai Public $70 \leq \text{skor} < 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Start	Public_Avg	Jika nilai Public $< 70 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Public_[Cabang]	Kreatif_Excel	Jika nilai Kreativitas $\geq 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Public_[Cabang]	Kreatif_Good	Jika nilai Kreativitas $70 \leq \text{skor} < 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Public_[Cabang]	Kreatif_Avg	Jika nilai Kreativitas $< 70 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Kreatif_[Cabang]	Tampil_Excel	Jika nilai Penampilan $\geq 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Kreatif_[Cabang]	Tampil_Good	Jika nilai Penampilan $70 \leq \text{skor} < 85 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0
Kreatif_[Cabang]	Tampil_Avg	Jika nilai Penampilan $< 70 \rightarrow$ Cost , selain itu $\rightarrow$ 999.0

Tampil_Excel	Finish	0.0 (Gerbang penutupan kalkulasi lintasan)
Tampil_Good	Finish	0.0 (Gerbang penutupan kalkulasi lintasan)
Tampil_Avg	Finish	0.0 (Gerbang penutupan kalkulasi lintasan)

Keterangan: Variabel Cost pada tabel dihitung menggunakan rumus penalti dasar berbobot: $\text{round}((100 - \text{Nilai}) * (\text{Bobot_Kriteria} / 100), 2)$.

BAB 4

IMPLEMENTASI

4.1 Implementasi Program

Implementasi program merupakan tahap di mana rancangan logika sistem diubah menjadi kode program nyata agar bisa dijalankan di komputer. Sistem Pendukung Keputusan (DSS) Penentuan Juara Lomba ini dibangun menggunakan bahasa pemrograman Python dengan bantuan framework Streamlit sebagai antarmuka berbasis web agar juri mudah memasukkan nilai.

Program ini menggunakan pendekatan teori graf. Setiap peserta lomba dinilai melalui jalur keputusan yang bercabang, kemudian sistem akan mencari rute dengan "biaya kerugian/penalti" (*cost*) yang paling kecil menggunakan Algoritma Dijkstra untuk menentukan siapa pemenang terbaiknya.

Berikut adalah beberapa pustaka (*library*) utama yang dipasang dan digunakan dalam program ini:

- **Streamlit (st):** Digunakan untuk membuat halaman web interaktif dengan cepat, seperti membuat tombol input, mengunggah file, menggeser bobot kriteria, dan menampilkan tabel hasil.
- **NetworkX (nx):** Pustaka khusus matematika graf untuk membuat titik keputusan (*node*), menghubungkan garis (*edge*), dan menyimpan nilai bobot penalti (*weight*).
- **Matplotlib (plt):** Digunakan untuk menggambar bagan jaringan graf secara visual ke layar komputer, seperti memberi warna pada titik yang dilewati peserta.
- **Pandas (pd):** Digunakan untuk membaca tabel data peserta (jika diunggah lewat file CSV) serta menyusun ranking juara dengan rapi.
- **Heapq:** Pustaka struktur data antrian yang membuat proses pencarian rute algoritma Dijkstra berjalan sangat cepat.

4.2 Penjelasan Kode

Kode program dibagi menjadi beberapa bagian utama agar mudah dipahami alur kerjanya:

4.2.1 Konfigurasi Halaman dan Pengisian Data

Pada bagian awal, program mengatur tampilan halaman web agar berbentuk melebar (*wide mode*) dan memberikan judul utama di layar. Sistem menyediakan dua pilihan bagi juri untuk memasukkan data peserta:

1. Ketik Manual: Juri memasukkan jumlah peserta, kemudian sistem membuat kolom teks kosong untuk diisi Nama, nilai *Public Speaking*, *Kreativitas*, dan *Penampilan* (skala 0-100).
2. Upload File CSV: Juri tinggal mengunggah file tabel .csv yang sudah disiapkan, lalu pustaka Pandas akan otomatis membaca seluruh data dalam hitungan detik.

4.2.2 Pengaturan Bobot Kriteria

Pada bagian ini, program menyediakan fitur interaktif yang memungkinkan tim juri untuk menentukan tingkat kepentingan atau prioritas dari masing-masing kriteria penilaian secara fleksibel. Fitur ini sangat penting karena dalam sebuah perlombaan, setiap kriteria sering kali memiliki bobot pengaruh yang tidak sama terhadap penentuan pemenang akhir.

Secara teknis, kode program memanfaatkan komponen *slider* dari framework Streamlit (`st.slider`). Komponen ini dipilih karena sangat ramah pengguna (*user-friendly*), di mana juri hanya perlu menggeser tombol ke kanan atau ke kiri untuk menentukan nilai persentase (mulai dari 0% hingga 100%) tanpa perlu mengetik angka secara manual. Terdapat tiga parameter bobot kriteria yang diatur dalam sistem ini, yaitu:

1. **Bobot Nilai Public Speaking (`b_public`):** Mengatur persentase pengaruh kemampuan berbicara di depan umum peserta, dengan nilai standar bawaan (*default*) sebesar 40%.
2. **Bobot Nilai Kreativitas (`b_kreativitas`):** Mengatur persentase pengaruh aspek ide dan inovasi peserta, dengan nilai standar bawaan sebesar 30%.
3. **Bobot Nilai Penampilan (`b_penampilan`):** Mengatur persentase pengaruh estetika dan pembawaan presentasi peserta, dengan nilai standar bawaan sebesar 30%.

Untuk mengoptimalkan pemanfaatan ruang pada layar monitor, ketiga *slider* tersebut diletakkan secara berdampingan secara horizontal dengan membagi halaman menjadi tiga kolom menggunakan fungsi `st.columns(3)`.

Selain menyediakan kontrol input, program ini juga dilengkapi dengan logika validasi otomatis untuk meminimalkan kesalahan manusia (*human error*) saat menentukan bobot. Berdasarkan aturan matematika keputusan, total akumulasi dari seluruh bobot kriteria harus berjumlah tepat 100% agar perhitungan kalkulasi skor akhir menjadi valid dan adil. Oleh karena itu, di dalam kode program disisipkan fungsi kondisi (`if`) sebagai pengingat:

```
Python ⬇️ 📄  
  
if b_public + b_kreativitas + b_penampilan != 100:  
    st.warning(f"Total bobot: {b_public + b_kreativitas + b_penampilan}%. Disarar
```

Cara kerja dari logika di atas sangat sederhana: setiap kali juri mengubah salah satu nilai *slider*, sistem akan langsung menjumlahkan ketiga bobot tersebut secara otomatis (*real-time*). Jika hasil penjumlahannya kurang dari 100% atau lebih dari 100%, sistem akan langsung mendeteksi ketidaksesuaian tersebut dan memunculkan kotak peringatan berwarna kuning di layar menggunakan fungsi `st.warning`.

Kotak peringatan ini berfungsi sebagai panduan interaktif bagi juri, yang menginformasikan jumlah bobot saat ini dan mengingatkan mereka untuk menyesuaikan kembali nilai *slider* hingga total akumulasinya mencapai angka 100%. Fitur validasi ini memastikan bahwa data bobot yang masuk ke tahap perhitungan Algoritma Dijkstra nantinya benar-benar akurat dan sah.

4.2.3 Fungsi Kategori dan Logika Penalti (*Cost*)

Sistem ini menggunakan logika terbalik yang disebut nilai penalti (*cost*). Algoritma rute terpendek (Dijkstra) bertugas mencari nilai paling kecil/pendek. Jadi, semakin bagus nilai asli peserta, maka nilai penalti yang diberikan sistem justru semakin kecil.

Sistem mengelompokkan kemampuan peserta ke dalam 3 tingkatan (Kategori):

- Excel (Sangat Bagus): Untuk nilai 85 sampai 100.
- Good (Bagus): Untuk nilai 70 sampai 84.
- Avg (Rata-rata/Cukup): Untuk nilai di bawah 70.

Rumus matematika sederhana yang digunakan di dalam kode adalah:

Penalti (Cost) = (100 - Nilai Riil Peserta) * Bobot Kriteria/100

4.2.4 Proses Kerja Algoritma Dijkstra dan Pembuatan Graf

Program menggunakan perulangan (for p in peserta:) untuk membuat satu peta graf berarah (*Directed Graph*) yang mandiri khusus bagi tiap peserta melalui pustaka NetworkX. Di dalam graf tersebut, terdapat 11 titik keputusan (*nodes*) yang tersusun sebagai berikut:

Start > 3 Cabang Public Speaking > 3 Cabang Kreativitas > 3 Cabang Penampilan > Finish

Agar Algoritma Dijkstra berjalan jujur sesuai nilai asli peserta, sistem menerapkan aturan penguncian rute:

- Jalur yang **sesuai** dengan nilai riil peserta akan diberikan bobot penalti asli (*cost*) hasil kalkulasi rumus kriteria.
- Jalur yang **tidak sesuai** akan otomatis dikunci dan diberikan nilai penalti raksasa sebesar **999.0**.

Karena sifat dasar Algoritma Dijkstra adalah mencari total rute terkecil/termurah, angka 999.0 ini berfungsi sebagai barikade virtual. Algoritma akan otomatis menolak jalur tersebut dan dipaksa hanya melewati satu-satunya jalur yang mencerminkan kemampuan asli peserta.

Setelah mencapai titik Finish, fungsi `dijkstra_pencatatan_jalur` akan menghitung total biaya penalti yang dikumpulkan. Guna mengubah nilai penalti tersebut menjadi bentuk nilai prestasi, program menggunakan rumus:

$$\text{Skor Akhir Rekomendasi} = 100 - \text{Total Penalty}$$

Semakin kecil penalti seorang peserta, maka skor akhirnya akan semakin mendekati 100. Seluruh hasil kalkulasi skor dan teks rute keputusan ini kemudian disimpan ke dalam *DataFrame* Pandas untuk diurutkan dari yang tertinggi guna menentukan peringkat juara.

4.2.5 Analisis Pintar (AI Feedback) dan Gambar Visualisasi

Sistem menggunakan logika komputer sederhana untuk memberikan masukan (*feedback*). Peserta dengan skor akhir tertinggi otomatis mendapatkan ucapan selamat di dalam kotak hijau sebagai Juara 1.

Bagi peserta yang belum juara, sistem akan otomatis mencari nilai kriteria mana yang paling rendah dan menuliskan saran otomatis di layar (misalnya: "*Disarankan fokus meningkatkan teknik Public Speaking*"). Di bagian paling bawah halaman, sistem menggambar grafik jaringan tersebut, di mana rute perjalanan nilai yang dipilih oleh algoritma Dijkstra akan ditebalkan dengan **warna hijau**, sementara jalan lain yang tidak terpilih diberi warna abu-abu samar.

4.3 Tampilan Sistem

Desain antarmuka (*User Interface*) aplikasi ini dirancang sangat ramah pengguna dengan membaginya ke dalam 4 bagian utama:

No	Nama Komponen Layar	Deskripsi Fungsi	Elemen Kode Streamlit
1	Menu Input Data	Tempat juri memilih cara memasukkan nama	st.radio, st.number_input, st.file_uploader
2	Geseran Bobot (%)	Tiga buah tombol geser untuk menentukan persentase nilai kriteria sesuai kesepakatan juri.	st.slider, st.columns

3	Tabel Ranking & AI Feedback	Menampilkan tabel urutan pemenang dari atas ke bawah, pengumuman juara 1, serta teks saran perbaikan otomatis.	st.dataframe, st.success, st.write
4	Gambar Visualisasi Graf	Menampilkan teks urutan stasiun keputusan yang dilewati beserta visualisasi gambar jaringan graf berwarna hijau-abu.	st.table, st.selectbox, st.pyplot

Melalui kombinasi antarmuka ini, juri tidak hanya melihat hasil akhir berupa angka mati, tetapi juga bisa melihat visualisasi transparan mengenai bagaimana sistem komputer menghitung dan memutuskan rekomendasi juara secara adil, objektif, dan akurat.

BAB 5

PENGUJIAN DAN ANALISIS

5.1 Skenario Pengujian

Di bagian ini, kita menguji aplikasi untuk memastikan sistem kalkulasi grafnya berjalan dengan benar dan tidak ada *error* saat dipakai oleh juri. Pengujian dibagi menjadi 3 kondisi:

1. Skenario 1: Pengujian Nilai Normal (*Data Ideal*)

Memastikan aplikasi bisa menghitung penalti rute graf dengan tepat dan bisa menentukan urutan juara dari nilai tertinggi ke terendah.

Input yang Dimasukkan:

- Bobot Kriteria: Public Speaking = 40%, Kreativitas = 30%, Penampilan = 30%
- Data Peserta:
 - **Peserta 1 (Nilai Bagus):** Public=90, Kreatif=88, Tampil=85 (Semua masuk kategori *Excel*)
 - **Peserta 2 (Nilai Sedang):** Public=75, Kreatif=80, Tampil=70 (Semua masuk kategori *Good*)
 - **Peserta 3 (Nilai Kurang):** Public=60, Kreatif=65, Tampil=55 (Semua masuk kategori *Avg*)

2. Skenario 2: Pengujian Jika Nilai Seri (*Tie-Breaker*)

Menguji apakah aplikasi bingung atau tidak jika ada dua peserta yang nilainya sama persis.

Input yang Dimasukkan:

- **Peserta A:** Public=80, Kreatif=80, Tampil=80
- **Peserta B:** Public=80, Kreatif=80, Tampil=80

3. Skenario 3: Pengujian Salah Input Bobot (*Error Handling*)

Memastikan aplikasi menolak melakukan hitungan jika juri salah memasukkan total bobot (misalnya totalnya malah 120%, bukan 100%).

5.2 Analisis Hasil Perhitungan

1. Hasil Pengujian Skenario 1 (Kesesuaian Jalur Graf)

Setelah tombol diproses, sistem berhasil menghitung nilai penalti (*cost*) dan rute untuk masing-masing peserta:

- **Peserta 1:** Nilainya besar, maka penaltinya kecil (Total Penalty = 12.1). Skor Akhir menjadi $100 - 12.1 = 87.90$.
- Jalur Graf yang Dilewati: Start → Public_Excel → Kreatif_Excel → Tampil_Excel → Finish

- **Hasil Akhir:** Aplikasi sukses menaruh **Peserta 1 sebagai Juara 1**. Jalur visualisasi graf (garis hijau tebal) juga menyala tepat di kategori "Excel". Sementara jalur salah lainnya otomatis dikunci menggunakan nilai penalti raksasa (9999.0) agar tidak dipilih oleh algoritma Dijkstra.

2. Hasil Pengujian Skenario 2 (Antisipasi Juara Kembar)

- Saat nilai Peserta A dan Peserta B dimasukkan sama persis, aplikasi tidak mengalami *error* atau langsung memilih salah satu secara acak.
- **Hasil Akhir:** Sistem memunculkan pesan sukses berwarna hijau: Rekomendasi Juara 1 (SERI): Peserta A, Peserta B. Ini membuktikan sistem sangat adil dan akurat dalam membaca situasi nilai kembar.

3. Hasil Pengujian Skenario 3 (Keamanan Sistem)

- Ketika dicoba memasukkan bobot yang tidak genap 100%, aplikasi langsung berhenti berhitung dan memunculkan peringatan warna merah. Aplikasi memaksa pengguna membetulkan bobot terlebih dahulu. Ini artinya, aplikasi aman dari kecerobohan juri (*human error*).

5.3 Kompleksitas Algoritma

Di bagian ini, kita mengukur seberapa cepat (waktu) dan seberapa hemat (memori) aplikasi ini saat dijalankan.

1. Kompleksitas Waktu (Kecepatan Aplikasi)

Aplikasi ini mencari jalur terpendek menggunakan **Algoritma Dijkstra** dibantu dengan struktur data *Min-Heap* (heapq) agar pencariannya super cepat.

- Di dalam matematika komputer, rumus kecepatan Dijkstra adalah:

$$O((V + E) \log V)$$

(di mana V adalah jumlah titik/node, dan E adalah jumlah garis/sisi graf).

- **Penjelasan Sederhananya:** Dalam aplikasi kita, jumlah titik (V) dan garis (E) pada graf itu **selalu tetap** (hanya ada 11 titik dan 21 garis untuk tiap peserta). Karena ukuran grafnya kecil dan tidak berubah, proses pencarian rute Dijkstra berjalan sangat instan (kurang dari 1 detik).
- Kecepatan aplikasi ini hanya dipengaruhi oleh seberapa banyak jumlah peserta (N) yang diinput. Jika pesertanya makin banyak, waktu prosesnya hanya akan bertambah secara lurus/linear. Jadi, kompleksitas waktunya secara nyata adalah **O(N)** (sangat cepat dan ringan).

2. Kompleksitas Ruang (Kehematan Memori RAM)

Kompleksitas ruang menghitung seberapa banyak memori komputer yang dipakai untuk menyimpan data graf peserta.

- **Penjelasan Sederhananya:** Karena graf untuk satu peserta ukurannya sangat kecil (11 titik dan 21 garis), memori RAM yang terpakai sangatlah sedikit.

- Berapa pun jumlah pesertanya (N), memori yang digunakan hanya akan naik sedikit secara linear mengikuti jumlah peserta tersebut. Rumus hemat memorinya adalah $O(N)$. Aplikasi ini dipastikan sangat ringan dan tidak akan membuat komputer atau server web menjadi lemot, bahkan jika digunakan untuk menilai ratusan peserta sekaligus.

BAB 6

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian yang telah dilakukan pada project akhir *Decision Support System* (DSS) Penentuan Juara Lomba berbasis Graph ini, dapat ditarik beberapa kesimpulan sebagai berikut:

1. Keberhasilan Pemodelan Struktur Graf Bercabang

Sistem telah berhasil mentransformasikan data penilaian multi-kriteria yang bersifat abstrak menjadi model **Multi-Layer Directed Graph Berorientasi Cabang (Multi-Path Layered Graph)**. Dengan membagi kriteria menjadi lapisan tingkatan kompetensi (Excel, Good, Avg) dan menggunakan formula biaya penalti (*cost penalty*), sistem mampu memetakan profil kemampuan riil peserta ke dalam struktur jaringan relasional secara akurat.

2. Efektivitas Algoritma Dijkstra sebagai Pengambil Keputusan

Algoritma Dijkstra yang dikombinasikan dengan teknik pemblokiran jalur ekstrem (bobot 999.0) teruji sangat efektif dan konsisten dalam mencari akumulasi biaya lintasan minimum (*shortest path*) dari node Start menuju node Finish. Peserta dengan total penalti terkecil (atau rute terpendek) secara otomatis terpilih sebagai rekomendasi Juara 1, sehingga menjamin proses keputusan yang objektif, transparan, dan bebas dari bias penilaian.

3. Fleksibilitas dan Kematangan Sistem

Aplikasi berbasis web yang dibangun menggunakan *framework* Streamlit berhasil menyajikan fungsionalitas yang interaktif dan adaptif. Fitur dwi-metode input (Ketik Manual dan Upload CSV) memudahkan juri dalam memproses variasi jumlah data peserta, sementara fitur *AI Smart Feedback* memberikan nilai tambah fungsional bagi peserta untuk mengevaluasi batasan kompetensi mereka.

6.2 Saran Pengembangan

Meskipun sistem DSS ini sudah berjalan dengan sangat baik dan memenuhi target spesifikasi project UAS Struktur Data, masih terdapat beberapa ruang improvisasi yang dapat dikembangkan di masa mendatang:

1. Penerapan Dynamic Node Thresholding

Pada sistem saat ini, batas nilai untuk penentuan cabang kompetensi (Excel $\geq$ 85, Good $\geq$ 70) masih bersifat statis tertulis di dalam kode program (*hardcoded*). Pengembangan berikutnya disarankan agar ambang batas (*threshold*) kategori tersebut dapat diatur secara dinamis oleh juri melalui antarmuka visual aplikasi sebelum kalkulasi graf dimulai.

2. Integrasi Pembobotan dengan Metode AHP (*Analytic Hierarchy Process*)

Penentuan persentase bobot kriteria pada sistem saat ini masih mengandalkan perkiraan subyektif juri via *slider*. Sistem dapat dikembangkan lebih ilmiah dengan

mengintegrasikan metode DSS lain seperti AHP untuk menghitung bobot prioritas kriteria secara otomatis berdasarkan matriks perbandingan berpasangan, sebelum nilainya diumpankan ke dalam bobot *edge* graf Dijkstra.

3. Penyimpanan Data Berbasis Graph Database

Untuk jangka panjang, disarankan untuk melakukan migrasi penyimpanan data dari yang semula bersifat lokal *runtime* Python (Pandas) menjadi terintegrasi dengan *Graph Database* sesungguhnya seperti **Neo4j**, agar aplikasi mampu mengelola histori rekam jejak ribuan peserta lintas periode kompetisi secara permanen dan terstruktur.